

Hackathon Assignment Round

You are invited to complete a hackathon-style assignment as part of the AasaMedChem recruitment process. The goal is to understand how you approach a small but realistic engineering problem.

High-level problem

Design and implement a small inventory and order management system using Next.js, a Neon-hosted PostgreSQL database, and deployment on Vercel.

Not every detail is fully specified on purpose. You are expected to make reasonable assumptions, design the flows, and document your decisions.

Core expectations

- Use Next.js for the application.
- Use Neon PostgreSQL as the primary database.
- Deploy the app on Vercel and provide a live URL.
- Implement authentication and role-based access for at least:
 - Admin
 - Seller/User (you can choose the exact terminology).

Inventory, units, and pricing

- Manage products with at least these quantity units:
 - grams (g)
 - kilograms (kg)
 - liters (L)
 - milliliters (mL)
 - items (unit/count)
- Prices should be displayed as INR amounts in the UI (for example, when showing product prices or quotation totals).
- Numeric fields (price, quantity, conversion factors, etc.) should handle:
 - High decimal precision
 - Large values (pick appropriate PostgreSQL types and explain your choices in the README).

You are free to extend the product model (for example, categories, SKUs, descriptions) as you see fit.

Unit conversion and storage strategy

- Choose a clear strategy for how quantities and prices are stored in the database.
- The exact approach is up to you, but it must be consistent and well documented.
- Document in the README:
 - How internal units are stored for different dimensions (weight, volume, count).
 - How prices and rates are stored internally.
 - Conversion factors (for example, kg $\leftrightarrow$ g, L $\leftrightarrow$ mL).

- Where and how conversions are applied in your code (for example, before saving, before display, during calculations).
- The UI should demonstrate that unit conversions are consistent and correct when viewing or placing orders.

Search, quotation, and ordering flow

Build a flow where a user can:

- Search and filter products.
- Select one or more products.
- Enter the quantity in any supported unit.
- See the calculated price based on the configured rate and the ordered quantity (even when the order unit differs from the internal storage unit).
- Place a quotation or order.

On the Admin side, it should be possible to:

- View incoming quotations/orders with:
 - Product details.
 - Units and quantities ordered.
 - Calculated pricing.
- See enough information to verify that conversions and prices are sensible.

Panels and features (guidelines, not strict prescriptions)

Treat the points below as guidelines. You can adjust, extend, or refine them if it leads to a better overall design, as long as the core flows exist.

Admin side could include, for example:

- Create, update, delete products.
- Configure base units and base prices.
- View inventory levels.
- View quotations/orders and their status.

Seller/User side could include, for example:

- Browse/search/filter products.
- See available units and prices.
- Place quotations/orders with flexible unit choice.

Technical and documentation requirements

- Use GitHub with meaningful, incremental commits (avoid a single “final” mega-commit).
- Use environment variables for secrets; do not commit credentials.
- Include a clear README that explains:
 - Project overview and features.
 - Tech stack and high-level system design (how frontend, backend, and database interact).
 - Database schema and key tables/fields with data types.

- Your unit storage and conversion strategy.
- How prices and quantities are stored (data types, precision, and any scaling/rounding rules).
- Setup instructions for running locally and connecting to Neon.
- How to deploy or re-deploy to Vercel.
- How to log in (test credentials) and how to use each panel and the quotation/order flow.

Evaluation (what will be looked at)

- Correctness of core flows (inventory, unit conversion, quotation/order, admin visibility).
- Thoughtfulness of data modelling and unit handling.
- Code structure and readability.
- Git commit history and workflow.
- Documentation and clarity of reasoning.
- UX decisions and how intuitive the overall system feels.

The detailed Google Form link and the exact time window for this test will be communicated to you. Your responses in the Google Form must be submitted within the given window.

Google Form link

https://docs.google.com/forms/d/e/1FAIpQLScj25yflLCjff_QHfjHkdWUnqlleABDfAZS0OX3HqtlgqmCvA/viewform?usp=header